

lecture_22

September 26, 2026

1 Lecture 22: Transportation Engineering Problems - IV

{note} Lecture 12's first motivating example previewed this problem: CMRL wants to set green-time splits at a signalized junction to minimize total vehicle delay, using a delay formula that is non-linear in the green time allocated to each phase. This lecture returns to that example in full - formulating the signal-timing problem as a constrained NLP (green times must sum to the available cycle length), solving it with the toolkit of Lectures 13-18, and using Lectures 19-20 to price the marginal value of green time itself.

Learning Objectives

By the end of this notebook, you will be able to: 1. Formulate a multi-phase signal-timing problem as a constrained NLP, using Webster-type delay functions and a cycle-length (effective green-time) constraint. 2. Solve signal-timing NLPs in Python via constrained-optimization solvers, handling both the cycle-length equality constraint and minimum-green inequality constraints. 3. Use KKT multipliers and the NLP Sensitivity Theorem to price the marginal value of one additional second of green time, and the marginal cost of minimum-green (pedestrian) requirements.

Prerequisites: NLP Principles (Lecture 12); Gradient Descent, Newton's Method, BFGS (Lectures 13-15); Penalty, Barrier, Interior-Point Methods (Lectures 16-18); NLP Sensitivity Analysis (Lecture 19); NLP Duality (Lecture 20); Facility Location (Lecture 21).

Estimated time: 90 minutes (including in-class exercises)

1.1 Why Signal Timing Optimization?

Lecture 12's first motivating example asked exactly this question:

- CMRL asks: *given the arrival flow and saturation flow on each approach to a signalized junction, how should the cycle's green time be split among phases to minimize total vehicle delay?* — an unconstrained-looking but genuinely constrained NLP, since every second of green time given to one phase is a second taken from another.
- The traffic engineer then asks: *given a pedestrian-crossing minimum-green requirement on a particular approach, how much extra vehicle delay does that safety requirement cost — and is a proposed signal-hardware upgrade that shortens the clearance interval worth it?* — a constrained NLP with an active minimum-green bound.

{note} This is a genuinely different flavour of non-linear transportation problem from Lectures 10-11's networks or Lecture 21's facility siting: the decision variables are time allocations at a single point (a junction), not flows or coordinates, and the natural constraint is a budget (available green time per cycle) rather than a network or geometric restriction. The same KKT machinery applies unchanged.

1.2 Notation

Symbol	Meaning
x_i	Decision variable — effective green time allocated to phase i (sec)
C	Cycle length (sec)
L	Total lost time per cycle (sec, start-up + clearance losses)
$G = C - L$	Effective green-time budget to be split among phases (sec)
λ_i	Degree-of-saturation (flow ratio) for phase i
μ_i	Saturation flow for phase i (veh/hr)
q_i	Arrival flow for phase i (veh/hr), used to weight per-vehicle delay into a total delay rate
$D_i(x_i)$	Webster-type average delay per vehicle on phase i (sec/veh), from Lecture 12

$$D_i(x_i) = \frac{C \left(1 - \frac{x_i}{C}\right)^2}{2 \left(1 - \frac{\lambda_i x_i}{C}\right)} + \frac{\lambda_i^2}{2\mu_i(1 - \lambda_i)}$$

The first (uniform-delay) term depends on the green time x_i ; the second (random/overflow-delay) term does not — it is a phase-specific constant once λ_i, μ_i are fixed.

1.3 The Signal-Timing Optimization Problem

1.3.1 Problem Statement

Given n phases at a signalized junction, each with a Webster delay function $D_i(x_i)$ and arrival flow q_i , and a fixed effective green-time budget $G = C - L$, find the green-time split that minimizes total vehicle delay.

1.3.2 NLP Formulation

Decision variables: $x_i \geq g_i^{\min}$ for each phase i (minimum green, e.g. pedestrian clearance).

Objective: minimize total delay rate,

$$\min_{\mathbf{x}} f(\mathbf{x}) = \sum_{i=1}^n q_i D_i(x_i)$$

Subject to: $\sum_{i=1}^n x_i = G, \quad x_i \geq g_i^{\min} \forall i$

{note} $D_i(x_i)$ is convex in x_i over the operating range $x_i < C/\lambda_i$ (delay blows up as the phase approaches oversaturation) - so this is a convex NLP, and the KKT point found by any of Lectures 16-18's methods is the global optimum.

1.4 In-Class Exercises

1.4.1 Exercise 1 — Two-Phase Signal Optimization

Problem Statement A CMRL-monitored junction runs a 90-second cycle with 10 seconds of lost time per cycle (80 sec of effective green to split between two phases):

Phase	λ_i	μ_i (veh/hr)	q_i (veh/hr)
Phase 1 (major road)	0.40	1,800	720
Phase 2 (minor road)	0.50	1,500	750

Find the green-time split that minimizes total vehicle delay.

Step 1 — Formulate the NLP

$$\min_{x_1, x_2} 720 D_1(x_1) + 750 D_2(x_2) \quad \text{s.t.} \quad x_1 + x_2 = 80, \quad x_1, x_2 \geq 10$$

Step 2 — Solve with `scipy.optimize.minimize(method='SLSQP')`

```
[ ]: # --- Exercise 1: two-phase signal-timing optimization ---

import numpy as np
from scipy.optimize import minimize, LinearConstraint

C, L = 90.0, 10.0
G = C - L    # 80 sec effective green budget

lam1, mu1, q1 = 0.40, 1800.0, 720.0
lam2, mu2, q2 = 0.50, 1500.0, 750.0

def D(x, lam, mu):
    return C * (1 - x / C) ** 2 / (2 * (1 - lam * x / C)) + lam ** 2 / (2 * mu * (1 - lam * x / C))
```

```

def dD(x, lam, mu, h=1e-6):
    return (D(x + h, lam, mu) - D(x - h, lam, mu)) / (2 * h)

def total_delay(x):
    return q1 * D(x[0], lam1, mu1) + q2 * D(x[1], lam2, mu2)

def grad_total(x):
    return np.array([q1 * dD(x[0], lam1, mu1), q2 * dD(x[1], lam2, mu2)])

eq = LinearConstraint([[1, 1]], [G], [G])
res = minimize(total_delay, x0=[G/2, G/2], jac=grad_total, method='SLSQP',
               constraints=[eq], bounds=[(10, G-10), (10, G-10)],
               options={'ftol': 1e-14})

x1s, x2s = res.x
print(f"Optimal split: x1* = {x1s:.4f} sec    x2* = {x2s:.4f} sec    (budget = {G:.4f} sec)")
print(f"D1(x1*) = {D(x1s, lam1, mu1):.4f} sec/veh    D2(x2*) = {D(x2s, lam2, mu2):.4f} sec/veh")
print(f"Total delay rate = {res.fun:,.2f} veh-sec/hr = {res.fun/3600:.4f} veh-hr/hr")
print(f"KKT multiplier (value of 1 sec of green budget) = {res.multipliers[0]:.4f} veh-sec/hr per sec")

```

Optimal split: x1* = 37.9938 sec x2* = 42.0062 sec (budget = 80 sec)
D1(x1*) = 18.0787 sec/veh D2(x2*) = 16.6922 sec/veh
Total delay rate = 25,535.82 veh-sec/hr = 7.0933 veh-hr/hr
KKT multiplier (value of 1 sec of green budget) = -430.9730 veh-sec/hr per sec

Step 3 — Sensitivity Analysis: The Value of One More Second of Green The multiplier's magnitude, 430.97 veh-sec/hr per second of effective green, is exactly what the NLP Sensitivity Theorem (Lecture 19) predicts: $\lambda^* = -\partial f^* / \partial G$. Verify numerically, and translate into a concrete engineering question: is a signal-hardware upgrade that shortens the clearance interval by 1 second (raising G from 80 to 81) worth its cost?

[1]: # --- Exercise 1: verifying the Sensitivity Theorem on the green-time budget ---

```

def solve_at_G(Gval, x0):
    eqG = LinearConstraint([[1, 1]], [Gval], [Gval])
    r = minimize(total_delay, x0, jac=grad_total, method='SLSQP',
                 constraints=[eqG], bounds=[(5, Gval-5), (5, Gval-5)],
                 options={'ftol': 1e-14})
    return r

res_p = solve_at_G(81.0, res.x)
res_m = solve_at_G(79.0, res.x)

```

```
f*(G=79) = 25,967.9932    f*(G=80) = 25,535.8210    f*(G=81) = 25,106.0549
Finite-diff df*/dG = -430.9692 => value of +1 sec green = 430.9692 veh-sec/hr
KKT multiplier from Step 2           = 430.9730 veh-sec/hr (matches)
```

Managerial insight: Every second of effective green recovered from lost time (e.g. via a faster-clearing signal head or a shorter all-red interval) is worth about 431 veh-sec/hr — roughly 1.2 vehicle-hours per peak hour, or 12 vehicle-hours over a 10-hour daily peak period. CMRL can now compare this concrete number against the capital cost of the signal-hardware upgrade, rather than relying on a qualitative “shorter cycles are better” argument.

Step 2 — Solve with `scipy.optimize.minimize(method='trust-constr')` (Lecture 18)

```
[ ]: # --- Exercise 2: three-phase junction with a binding minimum-green constraint
    ↪---

from scipy.optimize import NonlinearConstraint

C3, L3 = 120.0, 15.0
G3 = C3 - L3    # 105 sec
phases = [(0.30, 1900.0, 570.0), (0.55, 1700.0, 935.0), (0.45, 1600.0, 720.0)]

def D3(x, lam, mu):
    return C3 * (1 - x / C3) ** 2 / (2 * (1 - lam * x / C3)) + lam ** 2 / (2 *
    ↪mu * (1 - lam))

def dD3(x, lam, mu, h=1e-6):
    return (D3(x + h, lam, mu) - D3(x - h, lam, mu)) / (2 * h)

def total3(x):
    return sum(q * D3(xi, lam, mu) for xi, (lam, mu, q) in zip(x, phases))

def grad3(x):
    return np.array([q * dD3(xi, lam, mu) for xi, (lam, mu, q) in zip(x,
    ↪phases)])

eq3 = LinearConstraint([[1, 1, 1]], [G3], [G3])
gmin = 7.0
nlc_gmin = NonlinearConstraint(lambda x: x[0], gmin, np.inf)

x0 = [7.0, 67.18, 30.82]
res3 = minimize(total3, x0, jac=grad3, method='trust-constr', constraints=[eq3,
    ↪nlc_gmin],
                bounds=[(1, G3-2)]*3, options={'gtol': 1e-10, 'xtol': 1e-12,
    ↪'maxiter': 3000})

print(f"Optimal split: x1* = {res3.x[0]:.4f}  x2* = {res3.x[1]:.4f}  x3* =
    ↪{res3.x[2]:.4f} sec")
print(f"Total delay rate = {res3.fun:,.2f} veh-sec/hr")
print(f"Constraint x1 >= 7 active: x1* = {res3.x[0]:.4f}")

# res3.v is ordered exactly as constraints=[eq3, nlc_gmin] was passed in
mu_budget = res3.v[0][0]
nu_gmin = res3.v[1][0]
print(f"\nMultiplier of green-budget constraint (sum x = {G3:.0f}) : mu* =
    ↪{mu_budget:.4f} veh-sec/hr per sec")
print(f"Multiplier of minimum-green constraint (x1 >= 7)           : nu* = {nu_gmin:
    ↪.4f} veh-sec/hr per sec")
```

Optimal split: $x_1^* = 7.0000$ $x_2^* = 67.1843$ $x_3^* = 30.8157$ sec
 Total delay rate = 73,548.76 veh-sec/hr
 Constraint $x_1 \geq 7$ active: $x_1^* = 7.0000$

Multiplier of green-budget constraint (sum $x = 105$) : $\mu^* = 490.6304$ veh-sec/hr per sec
 Multiplier of minimum-green constraint ($x_1 \geq 7$) : $\nu^* = -22.8608$ veh-sec/hr per sec

Step 3 — Sensitivity Analysis: The Cost of the Pedestrian Minimum Without the pedestrian constraint, Phase 1's unconstrained-optimal green would fall to about 1.5 sec — far too short to clear a crossing safely. Quantify the trade-off by ranging $g_1^{\min}$.

```
[1]: # --- Exercise 2: sensitivity of total delay to the pedestrian minimum-green ---

def solve_gmin(gm, x0):
    nlc = NonlinearConstraint(lambda x: x[0], gm, np.inf)
    r = minimize(total3, x0, jac=grad3, method='trust-constr',
    ↪constraints=[eq3, nlc],
    ↪bounds=[(1, G3-2)]*3, options={'gtol': 1e-10, 'xtol': 1e-12,
    ↪'maxiter': 3000})
    return r

res_g6 = solve_gmin(6.0, res3.x)
res_g8 = solve_gmin(8.0, res3.x)
dfdgmin_fd = (res_g8.fun - res_g6.fun) / 2.0

print(f"f*(g_min=6) = {res_g6.fun:,.2f}    f*(g_min=7) = {res3.fun:,.2f}    ↪
    ↪f*(g_min=8) = {res_g8.fun:,.2f}")
print(f"Finite-diff df*/dg_min = {dfdgmin_fd:.4f} => |nu*| = {abs(dfdgmin_fd):
    ↪.4f} veh-sec/hr per sec")
print(f"Step 2's solver multiplier nu* = {nu_gmin:.4f} (matches in magnitude)")
print(f"\nCost of the current 7-sec pedestrian minimum, relative to an
    ↪unconstrained optimum: "
    ↪f"{res3.fun - 73485.58:.2f} veh-sec/hr")
```

$f*(g_{\min}=6) = 73,527.97$ $f*(g_{\min}=7) = 73,548.76$ $f*(g_{\min}=8) = 73,573.70$
 Finite-diff $df*/dg_{\min} = 22.8617$ $\Rightarrow |nu^*| = 22.8617$ veh-sec/hr per sec
 Step 2's solver multiplier $\nu^* = -22.8608$ (matches in magnitude)

Cost of the current 7-sec pedestrian minimum, relative to an unconstrained optimum: 63.18 veh-sec/hr

{caution} `trust-constr` returns one multiplier per constraint object, in exactly the order they were passed to `constraints=[...]` — here, `res3.v[0]` belongs to the green-budget equality and `res3.v[1]` belongs to the minimum-green inequality. Indexing into the wrong entry (an easy mistake with several active constraints) silently produces a plausible-looking but wrong number — 490.63

veh-sec/hr is a real, meaningful shadow price, just for the `*budget*` constraint, not the `*pedestrian-minimum*` constraint. Always confirm which number you have via a finite-difference re-solve, exactly as done above, before quoting either one to a decision-maker.

Step 4 — Interpretation

Managerial insight: The pedestrian-safety minimum on Phase 1 costs the junction about 22.9 veh-sec/hr for every additional second required — a small but non-zero price for crossing safety, and one that grows if pedestrian volumes later justify a longer minimum. The total cost of the current 7-second requirement (63.2 veh-sec/hr, roughly 1 vehicle losing an extra minute every hour) is a number CMRL can put directly next to the pedestrian-safety case when the two objectives are weighed against each other.

1.5 Take-Away Exercises

Work through each exercise following these steps: 1. Formulate the NLP (decision variables, weighted-delay objective, cycle-length constraint, any minimum-green constraints). 2. Solve using Python SciPy (SLSQP for equality-only, `trust-constr` when a minimum-green bound is active). 3. Verify any KKT multiplier reported by the solver against a finite-difference re-solve before interpreting it (Step 3's caution above). 4. Where indicated, carry out the requested sensitivity or duality analysis. 5. Record the optimal solution and write a one-paragraph managerial interpretation.

1.5.1 Exercise 1 — Two-Phase Junction

A BMTC-monitored junction runs a 100-second cycle with 12 seconds of lost time. Phase 1: $\lambda_1 = 0.35$, $\mu_1 = 1900$ veh/hr, $q_1 = 665$ veh/hr. Phase 2: $\lambda_2 = 0.45$, $\mu_2 = 1600$ veh/hr, $q_2 = 720$ veh/hr. Find the optimal green-time split.

1.5.2 Exercise 2 — Sensitivity to Cycle Length

Using this lecture's Exercise 1 junction, find how total delay changes if the cycle length itself is shortened from 90 to 80 seconds (with lost time held at 10 sec, so the effective green budget falls to 70 sec). Report the shadow price of the green-time budget at this new operating point, and compare it to the value found at $G = 80$. Is the marginal value of green time increasing or decreasing as the budget shrinks? Explain why, referencing the convexity of $D_i(x_i)$.

1.5.3 Exercise 3 — Three-Phase Junction with Two Active Minimums

A junction with a 110-second cycle and 14 seconds of lost time (96 sec effective green) serves three phases: Phase 1 ($\lambda = 0.25$, $\mu = 2000$, $q = 500$, min green 8 sec for a school crossing), Phase 2 ($\lambda = 0.50$, $\mu = 1750$, $q = 875$, min green 10 sec for a second crossing), Phase 3 ($\lambda = 0.40$, $\mu = 1600$, $q = 640$, no minimum beyond 5 sec). Solve with `trust-constr` and report which minimum-green constraints are active at the optimum.

1.5.4 Exercise 4 — Duality Verification (Python Implementation)

Using this lecture’s Exercise 1 two-phase problem: 1. Construct the Lagrangian dual $q(\mu) = \inf_{x_1, x_2} [q_1 D_1(x_1) + q_2 D_2(x_2) + \mu(G - x_1 - x_2)]$ by minimizing over (x_1, x_2) for a grid of fixed μ values. 2. Plot $q(\mu)$ using `matplotlib`, mark the μ^* found in Step 2 of this lecture’s Exercise 1, and confirm $q(\mu^*)$ equals the primal optimum. 3. This problem has only one constraint, so there is no indexing ambiguity like Exercise 2’s. Explain in 2-3 sentences why Exercise 2’s caution (matching `res.v[i]` to the right constraint) becomes more important, not less, as the number of simultaneously active constraints grows — and what habit (e.g. always finite-difference-checking before reporting) protects against the mistake regardless of how many constraints are present.

1.6 Circling Back

- Lecture 12 (NLP Principles): this lecture is the full realization of Lecture 12’s own first motivating example (CMRL green-time splits), extended from two phases to three and from unconstrained framing to an explicit cycle-length and minimum-green NLP.
- Lectures 16-18 (Penalty, Barrier, Interior-Point): Exercise 2’s minimum-green constraint is handled directly by `trust-constr`, recovering the multiplier as a by-product, exactly the workflow these lectures built — with a cautionary reminder that solver-reported multipliers still need finite-difference verification (Lecture 19’s own warning, reinforced here).
- Lecture 19 (NLP Sensitivity Analysis): both the green-time-budget shadow price and the pedestrian-minimum shadow price are verified numerically against finite differences, exactly as Lecture 19 recommends before quoting any multiplier as a managerial number.
- Lecture 21 (Facility Location): both lectures convert an abstract KKT multiplier into a concrete rupee-or-time-equivalent number a decision-maker can act on (SLA relaxation there, green-time or pedestrian-safety trade-off here).

1.7 Moving Forward

- This closes the exact-methods arc of the course (Lectures 12-22). The next module turns to non-exact NLP methods — local search, evolutionary computation, and swarm intelligence — for transportation problems (vehicle routing, network design) whose feasible regions are too irregular for gradient-based methods to handle directly.
-

1.8 Further Reading

- Webster, F.V. (1958). *Traffic Signal Settings*. Road Research Technical Paper No. 39, HMSO, London — origin of the Webster delay formula.
- Akcelik, R. (1981). *Traffic Signals: Capacity and Timing Analysis*. Australian Road Research Board — extensions to the Webster formula and oversaturated conditions.
- Roess, R.P., Prassas, E.S., and McShane, W.R. (2019). *Traffic Engineering* (5th ed.). Pearson — Chapter 9 (signal timing and delay).
- Nocedal, J. and Wright, S.J. (2006). *Numerical Optimization* (2nd ed.). Springer — Chapter 12 (KKT conditions, sensitivity, active-set behaviour).

- SciPy documentation: `scipy.optimize.minimize` with `method='SLSQP'/'trust-constr'`
— [docs.scipy.org](https://docs.scipy.org/doc/scipy/reference/optimize/minimize.html)